

6

READING AND ASSIGNING SECURITY DESCRIPTORS



In the previous chapter, we discussed the various structures that make up a security descriptor. You also learned how to manipulate security descriptors in PowerShell and how to represent them using the SDDL format. In this chapter, we'll discuss how to read security descriptors from kernel objects, as well as the more complex process of assigning security descriptors to these objects.

We'll focus our discussion on the security descriptors assigned to kernel objects. However, as mentioned in "Absolute and Relative Security Descriptors" on page 149, it's also possible to store a security descriptor in persistent storage, such as in a file or as a registry key value. In this case, the security descriptor must be stored in the relative format and read as a stream of bytes before we can convert it into a format we can inspect.

Reading Security Descriptors

To access a kernel object's security descriptor, you can call the `NtQuerySecurityObject` system call. This system call accepts a handle to the kernel object, as well as a set of flags that describe the components of the security descriptor you want to access. The `SecurityInformation` enumeration represents these flags.

Table 6-1 shows the list of available flags in the latest versions of Windows, as well as the location of the information in the security descriptor and the handle access required to query it.

Table 6-1: The SecurityInformation Flags and Their Required Access

Flag name	Description	Location	Handle access required
Owner	Query the owner SID.	Owner	ReadControl

Flag name	Description	Location	Handle access required
Group	Query the group SID.	Group	ReadControl
Dacl	Query the DACL.	DACL	ReadControl
Sacl	Query the SACL (auditing ACEs only).	SACL	AccessSystemSecurity
Label	Query the mandatory label.	SACL	ReadControl
Attribute	Query the system resource attribute.	SACL	ReadControl
Scope	Query the scoped policy ID.	SACL	ReadControl
ProcessTrustLabel	Query the process trust label.	SACL	ReadControl
AccessFilter	Query the access filter.	SACL	ReadControl
Backup	Query everything except the process trust label and access filter.	All	ReadControl and AccessSystemSecurity

You only need `ReadControl` access to read most of this information, except for the auditing ACEs from the SACL, which require `AccessSystemSecurity` access. (`ReadControl` access is sufficient for other ACEs stored in the SACL.)

The only way to get `AccessSystemSecurity` access is to first enable the `SeSecurityPrivilege` privilege, then explicitly request the access when opening a kernel object. [Listing 6-1](#) shows this behavior. You must run these commands as an administrator.

```
PS> $dir = Get-NtDirectory "\BaseNamedObjects" -Access AccessSystemSecurity
Get-NtDirectory : (0xC0000061) - A required privilege is not held by the client.
--snip--
```

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $dir = Get-NtDirectory "\BaseNamedObjects" -Access AccessSystemSecurity
PS> $dir.GrantedAccess
AccessSystemSecurity
```

Listing 6-1: Requesting AccessSystemSecurity access and enabling SeSecurityPrivilege

Our first attempt to open the BNO directory with `AccessSystemSecurity` access fails, because we don't have the required `SeSecurityPrivilege` privilege. Next, we enable that privilege and try again. This time we are able to open the directory, and printing its `GrantedAccess` parameter confirms we've been granted `AccessSystemSecurity` access.

It's not entirely clear why the designers of Windows made the decision to guard the reading of audit information with `SeSecurityPrivilege`. While we should consider modifying and removing audit information to be privileged actions, there is no obvious reason that reading that information should be. Unfortunately, we're stuck with this design.

You can query an object's security descriptor using the `Get-NtSecurityDescriptor` PowerShell command, which calls `NtQuerySecurityObject`. The system call returns the security descriptor in the relative format as a byte array, which the PowerShell command parses into a `SecurityDescriptor` object and returns to the caller. The command accepts either an object or a path to the resource you want to query, as shown in [Listing 6-2](#), which displays the security descriptor for the BNO directory.

```
PS> Use-NtObject($d = Get-NtDirectory "\BaseNamedObjects" -Access
ReadControl) {
    Get-NtSecurityDescriptor -Object $d
}
Owner                DACL ACE Count SACL ACE Count Integrity Level
-----
BUILTIN\Administrators 4                1                Low
```

Listing 6-2: Querying the security descriptor for the BNO directory

Here, we open the BNO directory with `ReadControl` access, then use `Get-NtSecurityDescriptor` to query the security descriptor from the open `Directory` object.

By default, the `Get-NtSecurityDescriptor` command queries for the owner, group, DACL, mandatory label, and process trust label. If you want to query any other field (or omit some of the returned information), you need to specify this through the `SecurityInformation` parameter, which accepts the values in [Table 6-1](#). For example, [Listing 6-3](#) uses a path instead of an object and requests only the `Owner` field.

```
PS> Get-NtSecurityDescriptor "\BaseNamedObjects" -SecurityInformation Owner
Owner                DACL ACE Count SACL ACE Count Integrity Level
```

BUILTIN\Administrators	NONE	NONE	NONE
------------------------	------	------	------

Listing 6-3: Querying the owner of the BNO directory

In the output, you can see that only the `Owner` column contains valid information; all other columns now have the value `NONE`, which indicates that no value is present, because we haven't requested that information.

Assigning Security Descriptors

Reading a security descriptor is easy; you just need the correct access to a kernel resource and the ability to parse the relative security descriptor format returned from the `NtQuerySecurityObject` system call. Assigning a security descriptor is a more complex operation. The security descriptor assigned to a resource depends on multiple factors:

- Is the resource being created?
- Did the creator specify a security descriptor during creation?
- Is the new resource stored in a container, such as a directory or registry key?
- Is the new resource a container or an object?
- What control flags are set on the parent or current security descriptor?
- What user is assigning the security descriptor?
- What ACEs does the existing security descriptor contain?
- What kernel object type is being assigned?

As you can see from the list, this process involves many variables and is one of the big reasons Windows security can be so complex.

We can assign a resource's security at creation time or via an open handle. Let's start with the more complex case first: assignment at creation time.

Assigning a Security Descriptor During Resource Creation

When creating a new resource, the kernel needs to assign it a security descriptor. Also, it must store the security descriptor differently depending on the kind of resource being created. For example, object manager resources are ephemeral, so the kernel will store their security descriptors in memory. In contrast, a filesystem driver's security descriptor must be persisted to disk; otherwise, it will disappear when you reboot your computer.

While the mechanism to store the security descriptor might differ, the kernel must still follow many common procedures when handling it, such as enforcing the rules of inheritance. To provide a consistent implementation, the kernel exports a couple of APIs that calculate the security descriptor to assign to a new resource. The most used of these APIs is `SeAssignSecurityEx`, which takes the following seven parameters:

Creator security descriptor An optional security descriptor on which to base the new assigned security descriptor

Parent security descriptor An optional security descriptor for the new resource’s parent object

Object type An optional GUID that represents the type of object being created

Container A Boolean value indicating whether the new resource is a container

Auto-inherit A set of bit flags that define the automatic inheritance behavior

Token A handle to the token to use as the creator’s identity

Generic mapping A mapping from generic access to specific access rights for the kernel type

Based on these parameters, the API calculates a new security descriptor and returns it to the caller. By investigating how these parameters interact, we can understand how the kernel assigns security descriptors to new objects.

Let’s consider this assignment process for a `Mutant` object. (This object will be deleted once the PowerShell instance closes, ensuring that we don’t accidentally leave unnecessary files or registry keys lying around.) [Table 6-2](#) provides an example of how we might set the parameters when creating a new `Mutant` object with `NtCreateMutant`.

Table 6-2: Example Parameters for a New Mutant Object

Parameter	Setting value
Creator security descriptor	The value of the <code>SecurityDescriptor</code> field in the object attributes structure.
Parent security descriptor	The security descriptor of the parent <code>Directory</code> ; not set for an unnamed <code>Mutant</code> .
Object type	Not set.
Container	Set to <code>False</code> , as a <code>Mutant</code> isn’t a container.
Auto-inherit	Set to <code>AutoInheritDacl</code> if the parent security descriptor’s control flags include the <code>DaclAutoInherited</code> flag and the creator DACL is missing or there is no creator security descriptor; set to <code>AutoInheritSacl</code> if the parent security descriptor’s control flags include the <code>SaclAutoInherited</code> flag

Parameter	Setting value
	and the creator SACL is missing or there is no creator security descriptor.
Token	If the caller is impersonating, set to an impersonation token; otherwise, set to the primary token of the caller's process.
Generic mapping	Set to the generic mapping for the <code>Mutant</code> type.

You might be wondering why the object type isn't set in [Table 6-2](#). The API supports the parameter, but neither the object manager nor the I/O manager uses it. Its primary purpose is to let Active Directory control inheritance, so we'll discuss it separately in "Determining Object Inheritance" on page 203.

[Table 6-2](#) shows only two possible auto-inherit flags, but we can pass many others to the API. [Table 6-3](#) lists the available auto-inherit flags, some of which we'll encounter in this chapter's examples.

Table 6-3: The Auto-inherit Flags

Flag name	Description
<code>DaclAutoInherit</code>	Auto-inherit the DACL.
<code>SaclAutoInherit</code>	Auto-inherit the SACL.
<code>DefaultDescriptorForObject</code>	Use the default security descriptor for the new security descriptor.
<code>AvoidPrivilegeCheck</code>	Don't check for privileges when setting the mandatory label or SACL.
<code>AvoidOwnerCheck</code>	Avoid checking whether the owner is valid for the current token.
<code>DefaultOwnerFromParent</code>	Copy the owner SID from the parent security descriptor.
<code>DefaultGroupFromParent</code>	Copy the group SID from the parent security descriptor.
<code>MacInNoWriteUp</code>	Auto-inherit the mandatory label with the <code>NoWriteUp</code> policy.
<code>MacInNoReadUp</code>	Auto-inherit the mandatory label with the <code>NoReadUp</code> policy.

Flag name	Description
Mac1NoExecuteUp	Auto-inherit the mandatory label with the NoExecuteUp policy.
AvoidOwnerRestriction	Ignore restrictions placed on the new DACL by the parent security descriptor.
ForceUserMode	Enforce all checks as if called from user mode (only applicable for kernel callers).

The most important `SeAssignSecurityEx` parameters to consider are the values assigned to the parent and creator security descriptors. Let's go through a few configurations of these two security descriptor parameters to understand the different outcomes.

Setting Only the Creator Security Descriptor

In the first configuration we'll consider, we call `NtCreateMutant` with the object attribute's `SecurityDescriptor` field set to a valid security descriptor. If the new `Mutant` object is not given a name, it will be created without a parent directory, and the corresponding parent security descriptor won't be set. If there is no parent security descriptor, the auto-inherit flags won't be set, either.

Let's test this behavior to see the security descriptor generated when we create a new `Mutant` object. Rather than creating the object itself, we'll use the user-mode implementation of `SeAssignSecurityEx`, which `NTDLL` exports as `RtlNewSecurityObjectEx`. We can access `RtlNewSecurityObjectEx` using the `New-NtSecurityDescriptor` PowerShell command, as shown in [Listing 6-4](#).

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
❶ PS> Add-NtSecurityDescriptorAce $creator -Name "Everyone" -Access GenericRead
❷ PS> Format-NtSecurityDescriptor $creator
Type: Mutant
Control: DaclPresent
<DACL>
  - Type   : Allowed
  - Name   : Everyone
  - SID    : S-1-1-0
  - Mask   : 0x80000000
  - Access : GenericRead
  - Flags  : None

PS> $token = Get-NtToken -Effective -Pseudo
❸ PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
-Type Mutant
PS> Format-NtSecurityDescriptor $sd
Type: Mutant
Control: DaclPresent
❹ <Owner>
  - Name   : GRAPHITE\user
  - Sid    : S-1-5-21-2318445812-3516008893-216915059-1002
```

```

❶ <Group>
  - Name   : GRAPHITE\None
  - Sid    : S-1-5-21-2318445812-3516008893-216915059-513

  <DACL>
  - Type   : Allowed
  - Name   : Everyone
  - SID    : S-1-1-0
  - Mask   : 0x00020001
❷ - Access: ModifyState|ReadControl
  - Flags  : None

```

Listing 6-4: Creating a new security descriptor from a creator security descriptor

We first build a creator security descriptor with only a single ACE, granting the *Everyone* group *GenericRead* access ❶. By formatting the security descriptor ❷, we can confirm that only the DACL is present in the formatted output. Next, using the creator security descriptor, we call the `New-NtSecurityDescriptor` command ❸, passing the current effective token and specifying the final object type as *Mutant*. This object type determines the generic mapping. Finally, we format the new security descriptor.

You might notice that the security descriptor has changed during the creation process: it has gained *Owner* ❹ and *Group* values ❺, and the specified access mask has changed from *GenericRead* to *ModifyState|ReadControl* ❻.

Let's start by considering where those new owner and group values come from. When we don't specify an *Owner* or *Group* value, the creation process copies these from the supplied token's *Owner* and *PrimaryGroup* SIDs. We can confirm this by checking the *Token* object's properties using the `Format-NtToken` PowerShell command, as shown in [Listing 6-5](#).

```

PS> Format-NtToken $token -Owner -PrimaryGroup
OWNER INFORMATION
-----
Name          Sid
----          ---
GRAPHITE\user S-1-5-21-2318445812-3516008893-216915059-1002

PRIMARY GROUP INFORMATION
-----
Name          Sid
----          ---
GRAPHITE\None S-1-5-21-2318445812-3516008893-216915059-513

```

Listing 6-5: Displaying the *Owner* and *PrimaryGroup* SIDs for the current effective token

If you compare the output in [Listing 6-5](#) with the security descriptor values in [Listing 6-4](#), you can see that the owner and group SIDs match.

In [Chapter 4](#), you learned that it's not possible to set an arbitrary owner SID on a token; this value must be either the user's SID or a SID marked with the *Owner* flag. You might wonder: As the token's SID is being used to set the security descriptor's default owner, can we use this behavior to

specify an arbitrary owner SID in the security descriptor? Let's check. In [Listing 6-6](#), we first set the security descriptor to the *SYSTEM* user's SID, then try to create the security descriptor again.

```
PS> Set-NtSecurityDescriptorOwner $creator -KnownSid LocalSystem
PS> New-NtSecurityDescriptor -Token $token -Creator $creator -Type Mutant
New-NtSecurityDescriptor : (0xC000005A) - Indicates a particular Security ID
may not be assigned as the owner of an object.
```

Listing 6-6: Setting the SYSTEM user as the Mutant object's security descriptor owner

This time, the creation fails with an exception and the status code `STATUS_INVALID_OWNER`. This is because the API checks whether the owner SID being assigned is valid for the supplied token. It doesn't have to be the `Token` object's owner SID, but it must be either the user's SID or a group SID with the `Owner` flag set.

You can set an arbitrary owner SID only when the token used to create the security descriptor has the `SeRestorePrivilege` privilege enabled. Note that this token doesn't necessarily have to belong to the caller of the `SeAssignSecurityEx` API. You can also disable the owner check by specifying the `AvoidOwnerCheck` auto-inherit flag; however, the kernel will never specify this flag when creating a new object, so it will always enforce the owner check.

This is not to say that there's no way to set a different owner as a normal user. However, any method of setting an arbitrary owner that you discover is a security vulnerability that Microsoft will likely fix. An example of such a bug is CVE-2018-0748, which allowed users to set an arbitrary owner when creating a file. The user had to create the file via a local filesystem share, causing the owner check to be bypassed.

There are no restrictions on the value of the group SID, as the group doesn't contribute to the access check. However, restrictions apply to the SACL. If you specify any audit ACEs in the SACL as part of the creator security descriptor, the kernel will require `SeSecurityPrivilege`.

Remember that when we created the security descriptor, the access mask changed? This is because the security descriptor assignment process maps all generic access rights in the access mask to type-specific access rights using the object type's generic mapping information. In this case, the `Mutant` type's `GenericRead` mapping converts the access mask to `ModifyState|ReadControl`. There is one exception to this rule: if the ACE has the `InheritOnly` flag set, then generic access rights won't be mapped. You'll understand why the exception exists shortly, when we discuss inheritance.

We can confirm this mapping behavior by using `New-NtSecurityDescriptor` to create an unnamed `Mutant` object, as shown in [Listing 6-7](#).

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $creator -Name "Everyone" -Access GenericRead
```

```

PS> Use-NtObject($m = New-NtMutant -SecurityDescriptor $creator) {
    Format-NtSecurityDescriptor $m
}
Type: Mutant
Control: DaclPresent
<Owner>
- Name : GRAPHITE\user
- Sid : S-1-5-21-2318445812-3516008893-216915059-1002

<Group>
- Name : GRAPHITE\None
- Sid : S-1-5-21-2318445812-3516008893-216915059-513

<DACL>
- Type : Allowed
- Name : Everyone
- SID : S-1-1-0
- Mask : 0x00020001
- Access: ModifyState|ReadControl
- Flags : None

```

Listing 6-7: Verifying security descriptor assignment rules by creating a Mutant object

As you can see, the output security descriptor is the same as the one created in [Listing 6-4](#).

Setting Neither the Creator nor the Parent Security Descriptor

Let's explore another simple case. In this scenario, neither the creator nor the parent security descriptor is set. This case corresponds to calling `NtCreateMutant` without a name or a specified `SecurityDescriptor` field. The script to test it is even simpler than the previous one, as shown in [Listing 6-8](#).

```

PS> $token = Get-NtToken -Effective -Pseudo
❶ PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -HideHeader
❷ <Owner>
- Name : GRAPHITE\user
- Sid : S-1-5-21-2318445812-3516008893-216915059-1002

<Group>
- Name : GRAPHITE\None
- Sid : S-1-5-21-2318445812-3516008893-216915059-513

❸ <DACL>
- Type : Allowed
- Name : GRAPHITE\user
- SID : S-1-5-21-2318445812-3516008893-216915059-1002
- Mask : 0x001F0001
- Access: Full Access
- Flags : None

- Type : Allowed
- Name : NT AUTHORITY\SYSTEM
- SID : S-1-5-18
- Mask : 0x001F0001

```

```

- Access: Full Access
- Flags : None

- Type : Allowed
- Name : NT AUTHORITY\LogonSessionId_0_137918
- SID : S-1-5-5-0-137918
- Mask : 0x00120001
- Access: ModifyState|ReadControl|Synchronize
- Flags : None

```

Listing 6-8: Creating a new security descriptor with no creator or parent security descriptor

This call to `New-NtSecurityDescriptor` requires only the token and kernel object type ❶. The `Owner` and `Group` fields in the final security descriptor are set to default values based on the token's `Owner` and `PrimaryGroup` properties ❷.

But where did the DACL ❸ come from? We haven't specified either a parent or a creator security descriptor, so it couldn't have come from either of those. Instead, it's based on the `Token` object's *default DACL*, an ACL stored in the token that acts as a fallback when there is no other DACL specified. You can display a token's default DACL by passing the token to `Format-NtToken` with the `DefaultDac1` parameter, as in [Listing 6-9](#).

```

PS> Format-NtToken $token -DefaultDac1
DEFAULT DACL
-----
GRAPHITE\user: (Allowed)(None)(GenericAll)
NT AUTHORITY\SYSTEM: (Allowed)(None)(GenericAll)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(GenericExecute|GenericRead)

```

Listing 6-9: Displaying a token's default DACL

Other than its `Mutant`-specific access rights, the DACL in [Listing 6-9](#) matches the one in [Listing 6-8](#). We can conclude that, if we specify neither the parent nor the creator security descriptor during creation, we'll create a new security descriptor based on the token's owner, primary group, and default DACL. However, just to be certain, let's verify this behavior by creating an unnamed `Mutant` with no security descriptor ([Listing 6-10](#)).

```

PS> Use-NtObject($m = New-NtMutant) {
    Format-NtSecurityDescriptor $m
}
Type: Mutant
Control: None
<NO SECURITY INFORMATION>

```

Listing 6-10: Creating an unnamed Mutant to verify the default security descriptor creation behavior

Wait—the new `Mutant` object has no security information at all! That's not what we expected.

The issue here is that the kernel allows certain object types to have no security when the object doesn't have a name. You can learn whether an object requires security by querying its `SecurityRequired` property, as shown in [Listing 6-11](#).

```
PS> Get-NtType "Mutant" | Select-Object SecurityRequired
SecurityRequired
-----
False
```

Listing 6-11: Querying for the Mutant type's SecurityRequired property

As you can see, the `Mutant` type doesn't require security. So, if we specify neither the creator nor the parent security descriptor when creating an unnamed `Mutant` object, the kernel won't generate a default security descriptor.

Why would the kernel support the ability to create an object without a security descriptor? Well, if applications won't share that object with each other, the security descriptor would serve no purpose; it would only use up additional kernel memory. Only if you created an object with a name, so that it can be shared, would the kernel require security.

DUPLICATING UNNAMED OBJECT HANDLES

You can duplicate a handle to an unnamed resource and share it with another process without giving the resource a name. However, this should be done with care. While handle duplication allows you to remove access from a handle if the object has no security descriptor, the receiving process can easily reduplicate the handle to retrieve the access that was removed.

Prior to Windows 8, there was no way to assign security to an unnamed object that had `SecurityRequired` set to `False`. This has changed, and if you specify a security descriptor during creation, you'll assign it to the resulting object.

Windows 8 also introduced a new, undocumented flag to `NtDuplicateObject` to separately deal with the issue. Specifying the `NoRightsUpgrade` flag while duplicating a handle tells the kernel to deny any further duplication operations that request additional access rights.

To verify the generation of a default security descriptor, let's now create an object that requires security, such as a `Directory` object ([Listing 6-12](#)).

```
PS> Get-NtType Directory | Select-Object SecurityRequired
SecurityRequired
-----
True

PS> Use-NtObject($dir = New-NtDirectory) {
    Format-NtSecurityDescriptor $dir -Summary
}
GRAPHITE\user: (Allowed)(None)(Full Access)
```

```
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(Query|Traverse|ReadControl)
```

Listing 6-12: Creating an unnamed Directory to verify the default security descriptor

Listing 6-12 shows that the default security descriptor matches our assumptions.

Setting Only the Parent Security Descriptor

The next case we'll consider is much more complex. Say we call `NtCreateMutant` with a name but without specifying the `SecurityDescriptor` field. Because a named `Mutant` must be created within a `Directory` object (which, as we've just seen, requires security), the parent security descriptor will be set.

Yet when we specify a parent security descriptor, we also bring something else into play: *inheritance*, a process by which the new security descriptor copies a part of the parent security descriptor. Inheritance rules determine which parts of the parent get passed to the new security descriptor, and we call a parent security descriptor *inheritable* if its parts can be inherited.

The purpose of inheritance is to define a hierarchical security configuration for a tree of resources. Without inheritance, we would have to explicitly assign a security descriptor for each new object in the hierarchy, which would become unmanageable rather quickly. It would also make the resource tree impossible to manage, as each application might choose to behave differently.

Let's test the inheritance rules that apply when we create new kernel resources. We'll focus on the DACL, but these concepts apply to the SACL, as well. To minimize code duplication, **Listing 6-13** defines a few functions that run a test with the parent security descriptor and implement various options.

```
PS> function New-ParentSD($AceFlags = 0, $Control = 0) {
    $owner = Get-NtSid -KnownSid BuiltinAdministrators
    ❶ $parent = New-NtSecurityDescriptor -Type Directory -Owner $owner -Group $owner
    ❷ Add-NtSecurityDescriptorAce $parent -Name "Everyone" -Access GenericAll
    Add-NtSecurityDescriptorAce $parent -Name "Users" -Access GenericAll -Flags $AceFlags
    ❸ Add-NtSecurityDescriptorControl $parent -Control $Control
    ❹ Edit-NtSecurityDescriptor $parent -MapGeneric
    return $parent
}

PS> function Test-NewSD($AceFlags = 0,
    $Control = 0,
    $Creator = $null,
    [switch]$Container) {
    ❺ $parent = New-ParentSD -AceFlags $AceFlags -Control $Control
    Write-Output "-- Parent SD --"
    Format-NtSecurityDescriptor $parent -Summary
```

```

    if ($Creator -ne $null) {
        Write-Output "`r`n-= Creator SD =-"
        Format-NtSecurityDescriptor $creator -Summary
    }

    ❶ $auto_inherit_flags = @()
    if (Test-NtSecurityDescriptor $parent -DaclAutoInherited) {
        $auto_inherit_flags += "DaclAutoInherit"
    }
    if (Test-NtSecurityDescriptor $parent -SaclAutoInherited) {
        $auto_inherit_flags += "SaclAutoInherit"
    }
    if ($auto_inherit_flags.Count -eq 0) {
        $auto_inherit_flags += "None"
    }

    $token = Get-NtToken -Effective -Pseudo
    ❷ $sd = New-NtSecurityDescriptor -Token $token -Parent $parent -Creator $creator -Type Mutant -C
    Write-Output "`r`n-= New SD =-"
    ❸ Format-NtSecurityDescriptor $sd -Summary
}

```

Listing 6-13: Test function definitions for New-ParentSD and Test-NewSD

The `New-ParentSD` function creates a new security descriptor with the `Owner` and `Group` fields set to the `Administrators` group ❶. This will allow us to check for inheritance of the `Owner` or `Group` field in any new security descriptor we create from this parent. We also set the `Type` to `Directory`, as expected for the object manager. Next, we add two `Allowed` ACEs, one for the `Everyone` group and one for the `Users` group ❷, differentiated by their SIDs. We assign both ACEs `GenericAll` access and add some extra flags for the `Users` ACE.

The function then sets some optional security descriptor control flags ❸. Normally, when we assign a security descriptor to a parent the generic access rights get mapped to type-specific access rights. Here, we use `Edit-NtSecurityDescriptor` with the `MapGeneric` parameter to do this mapping for us ❹.

In the `Test-NewSD` function, we create the parent security descriptor ❶ and calculate any auto-inherit flags ❷. Then we create a new security descriptor, setting the `Container` property if required, as well as the auto-inherit flags we calculated ❸. You can specify a creator security descriptor for this function to use to create the new security descriptor. For now, we'll leave this value as `$null`, but we'll come back to it in the next section. Finally, we print the parent, the creator (if specified), and the new security descriptors to the console to verify the input and output ❹.

Let's start by testing the default case: running the `Test-NewSD` command with no additional parameters. The command will create a parent security descriptor with no control flags set, so there should be no auto-inherit flags present in the call to `SeAssignSecurityEx` ([Listing 6-14](#)).

```

PS> Test-NewSD
-= Parent SD =-

```

```

<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(None)(Full Access)

-= New SD -=
<Owner> : GRAPHITE\user ❶
<Group> : GRAPHITE\None
<DACL>
GRAPHITE\user: (Allowed)(None)(Full Access) ❷
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(ModifyState|ReadControl|...)

```

Listing 6-14: Creating a new security descriptor with a parent security descriptor and no creator security descriptor

In the output, we can see that the `Owner` and `Group` do not derive from the parent security descriptor ❶; instead, they're the defaults we observed earlier in this chapter. This makes sense: the caller, and not the user who created the parent object, should own the new resource.

However, the new DACL doesn't look as we might have expected ❷. It's set to the default DACL we saw earlier, and it bears no relation to the DACL we built in the parent security descriptor. The reason we didn't get any ACEs from the parent's DACL is that we did not specify the ACEs as inheritable. To do so, we need to set one or both of the `ObjectInherit` and `ContainerInherit` ACE flags. The former applies only to non-container objects such as `Mutant` objects, while the latter applies to container objects such as `Directory` objects. The distinction between the two types is important, because they affect how the inherited ACEs propagate to child objects.

The `Mutant` object is a non-container, so let's add the `ObjectInherit` flag to the ACE in the parent security descriptor ([Listing 6-15](#)).

```

PS> Test-NewSD -AceFlags "ObjectInherit" ❶
-= Parent SD -=
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit)(Full Access)

-= New SD -=
<Owner> : GRAPHITE\user ❷
<Group> : GRAPHITE\None
<DACL> ❸
BUILTIN\Users: (Allowed)(None)(ModifyState|Delete|ReadControl|WriteDac|WriteOwner)

```

Listing 6-15: Adding an `ObjectInherit` ACE to the parent security descriptor

In this listing, we specify the `ObjectInherit` ACE flag to the test function ❶. Observe that the `Owner` and `Group` fields have not changed ❷, but the DACL is no longer the default ❸. Instead, it contains a single ACE that grants the

`Users` group `ModifyState|Delete|ReadControl|WriteDac|WriteOwner` access. This is the ACE that we set to be inherited.

However, you might notice a problem: the parent security descriptor's ACE was granted `Full Access`, while the new security descriptor's ACE is not. Why has the access mask changed? In fact, it hasn't; the inheritance process has merely taken the raw `Directory` access mask for the parent security descriptor's ACE (the value `0x000F000F`) and copied it to the inherited ACE. A `Mutant` object's valid access bits are `0x001F0001`. Therefore, the inheritance process uses the closest mapping, `0x000F0001`, as shown in [Listing 6-16](#).

```
PS> Get-NtAccessMask (0x0001F0001 -band 0x000F000F) -ToSpecificAccess Mutant
ModifyState, Delete, ReadControl, WriteDac, WriteOwner
```

Listing 6-16: Checking the inherited access mask

This is a pretty serious issue. Notice, for example, that the `Mutant` type is missing the `Synchronize` access right, which it needs for a caller to wait on the lock. Without this access, the `Mutant` object would be useless to an application.

We can solve this access mask problem by specifying a generic access mask in the ACE. This will map to a type-specific access mask when the new security descriptor is created. There is only one complication: we've taken the parent security descriptor from an existing object, so the generic access was already mapped when the security descriptor was assigned. We simulated this behavior in our test function with the `Edit-NtSecurityDescriptor` call.

To resolve this issue, the ACE can set the `InheritOnly` flag. As a result, any generic access will remain untouched during the initial assignment. The `InheritOnly` flag marks the ACE for inheritance only, which prevents the generic access from being an issue for access checking. In [Listing 6-17](#), we check this behavior by modifying the call to the test function.

```
❶ PS> Test-NewSD -AceFlags "ObjectInherit, InheritOnly"
    -= Parent SD =-
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    <DACL>
    Everyone: (Allowed)(None)(Full Access)
❷ BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

    -= New SD =-
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DACL>
❸ BUILTIN\Users: (Allowed)(None)(Full Access)
```

Listing 6-17: Adding an InheritOnly ACE

In this listing, we change the ACE flags to `ObjectInherit` and `InheritOnly` ❶.

In the parent security descriptor's output, we can see that the access mask is no longer mapped from `GenericAll` ❷. As a result, the inherited ACE is now granted `Full Access`, as we require ❸.

Presumably, the `ContainerInherit` flag works in the same way as `ObjectInherit`, right? Not quite. We test its behavior in [Listing 6-18](#).

```
❶ PS> Test-NewSD -AceFlags "ContainerInherit, InheritOnly" -Container
    -= Parent SD -=
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    <DAcl>
    Everyone: (Allowed)(None)(Full Access)
    BUILTIN\Users: (Allowed)(ContainerInherit, InheritOnly)(GenericAll)

    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DAcl>
❷ BUILTIN\Users: (Allowed)(None)(Full Access)
❸ BUILTIN\Users: (Allowed)(ContainerInherit, InheritOnly)(GenericAll)
```

Listing 6-18: Creating a new security descriptor with the `ContainerInherit` flag

Here, we add the `ContainerInherit` and `InheritOnly` flags to the ACE and then pass the function the `Container` parameter ❶. Unlike in the `ObjectInherit` case, we now end up with two ACEs in the DACL. The first ACE ❷ grants access to the new resource based on the inheritable ACE. The second ❸ is a copy of the inheritable ACE, with `GenericAll` access.

NOTE

You might wonder how we can create a security descriptor for a container type when we're using the `Mutant` type. The answer is that the API doesn't care about the final type, as it uses only the generic mapping; when creating a real `Mutant` object, however, the kernel would never specify the `Container` flag.

The ACE's automatic propagation is useful, as it allows you to build a hierarchy of containers without needing to manually grant them access rights. However, you might sometimes want to disable this automatic propagation by specifying the `NoPropagateInherit` ACE flag, as shown in [Listing 6-19](#).

```
PS> $ace_flags = "ContainerInherit, InheritOnly, NoPropagateInherit"
PS> Test-NewSD -AceFlags $ace_flags -Container
--snip--
    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DAcl>
❶ BUILTIN\Users: (Allowed)(None)(Full Access)
```

Listing 6-19: Using `NoPropagateInherit` to prevent the automatic inheritance of ACEs

When we specify this flag, the ACE that grants access to the resource remains present, but the inheritable ACE disappears ❶.

Let’s try another ACE flag configuration to see what happens to `ObjectInherit` ACEs when they’re inherited by a container (Listing 6-20).

```
PS> Test-NewSD -AceFlags "ObjectInherit" -Container
--snip--
-= New SD -=
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
❶ BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(ModifyState|...)
```

Listing 6-20: Testing the `ObjectInherit` flag on a container

You might not expect the container to inherit the ACE at all, but in fact, it receives the ACE with the `InheritOnly` flag automatically set ❶. This allows the container to pass the ACE to non-container child objects.

Table 6-4 summarizes the inheritance rules for container and non-container objects based on the parent ACE flags. Objects are bolded where no inheritance occurs.

Table 6-4: Parent ACE Flags and Flags Set on the Inherited ACEs

Parent ACE flags	Non-container object	Container object
None	No inheritance	No inheritance
ObjectInherit	None	InheritOnly ObjectInherit
ContainerInherit	No inheritance	ContainerInherit
ObjectInherit NoPropagateInherit	None	No inheritance
ContainerInherit NoPropagateInherit	No inheritance	None
ContainerInherit ObjectInherit	None	ContainerInherit ObjectInherit
ContainerInherit ObjectInherit NoPropagateInherit	None	None

Finally, consider *auto-inherit* flags. If you return to Table 6-3, you can see that if the DACL has the `DacLAutoInherited` control flag set, the kernel will

pass the `DaclAutoInherit` flag to `SeAssignSecurityEx`, as there is no creator security descriptor. (The SACL has a corresponding `SaclAutoInherit` flag, but we'll focus on the DACL here.) What does the `DaclAutoInherit` flag do? In

[Listing 6-21](#), we perform a test to find out.

```
PS> $ace_flags = "ObjectInherit, InheritOnly"
❶ PS> Test-NewSD -AceFlags $ace_flags -Control "DaclAutoInherited"
    -= Parent SD -=
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    ❷ <DACL> (Auto Inherited)
    Everyone: (Allowed)(None)(Full Access)
    BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    ❸ <DACL> (Auto Inherited)
    ❹ BUILTIN\Users: (Allowed)(Inherited)(Full Access)
```

Listing 6-21: Setting the `DaclAutoInherited` control flag in the parent security descriptor

We set the parent security descriptor's control flags to contain the `DaclAutoInherited` flag ❶, and we confirm that it's set by looking at the formatted DACL ❷. We can see that the new security descriptor contains the flag as well ❸; also, the inherited ACE has the `Inherited` flag ❹.

How do the auto-inherit flags differ from the inheritance flags we discussed earlier? Microsoft conserves both inheritance types for compatibility reasons (as it didn't introduce the `Inherited` flag until Windows 2000). From the kernel's perspective, the two types of inheritance are not very different other than determining whether the new security has the `DaclAutoInherited` flag set and whether any inherited ACE gets the `Inherited` flag. But from a user-mode perspective, this inheritance model indicates which parts of the DACL were inherited from a parent security descriptor. That's important information, and various Win32 APIs use it, as we'll discuss in "Win32 Security APIs" on page 208.

Setting Both the Creator and Parent Security Descriptors

In the final case, we call `NtCreateMutant` with a name and specify the `SecurityDescriptor` field, setting both the creator and parent security descriptor parameters. To witness the resulting behavior, let's define some test code. [Listing 6-22](#) writes a function to generate a creator security descriptor. We'll reuse the `Test-NewSD` function we wrote earlier to run the test.

```
PS> function New-CreatorSD($AceFlags = 0, $Control = 0, [switch]$NoDacl) {
    ❶ $creator = New-NtSecurityDescriptor -Type Mutant
    ❷ if (!$NoDacl) {
        ❸ Add-NtSecurityDescriptorAce $creator -Name "Network" -Access GenericAll
        Add-NtSecurityDescriptorAce $creator -Name "Interactive"
    }
    -Access GenericAll -Flags $AceFlags
}
```

```

    }
    Add-NtSecurityDescriptorControl $creator -Control $Control
    Edit-NtSecurityDescriptor $creator -MapGeneric
    return $creator
}

```

Listing 6-22: The New-CreatorSD test function

This function differs from the `New-ParentSD` function created in [Listing 6-13](#) in the following ways: we use the `Mutant` type when creating the security descriptor ❶, we allow the caller to not specify a DACL ❷, and we set a different SID for the DACL if it is used ❸. These changes will allow us to distinguish the parts of a new security descriptor that come from the parent and those that come from the creator.

In some simple cases, the parent security descriptor has no inheritable DACL, and the API follows the same rules it uses when only the creator security descriptor is set. In other words, if the creator specifies the DACL, the new security descriptor will use it. Otherwise, it will use the default DACL.

If the parent security descriptor contains an inheritable DACL, the new security descriptor will inherit it, unless the creator security descriptor also has a DACL. Even an empty or NULL DACL will override the inheritance from the parent. In [Listing 6-23](#), we verify this behavior.

```

❶ PS> $creator = New-CreatorSD -NoDac1
❷ PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
    -= Parent SD -=
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    <DACL>
    Everyone: (Allowed)(None)(Full Access)
❸ BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

    -= Creator SD -=
❹ <NO SECURITY INFORMATION>

    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DACL>
❺ BUILTIN\Users: (Allowed)(None)(Full Access)

```

Listing 6-23: Testing parent DACL inheritance with no creator DACL

We first build a creator security descriptor with no DACL ❶, then run the test with an inheritable parent security descriptor ❷. In the output, we confirm the inheritable ACE for the `Users` group ❸ and that the creator has no DACL set ❹. When we create the new security descriptor, it receives the inheritable ACE ❺.

Let's also check what happens when we set a creator DACL ([Listing 6-24](#)).

```

❶ PS> $creator = New-CreatorSD
❷ PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
    -= Parent SD -=
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    <DACL>
    Everyone: (Allowed)(None)(Full Access)
    BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

    -= Creator SD -=
    <DACL>
    NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
    NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DACL>
❸ NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
   NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

```

Listing 6-24: Testing the overriding of parent DACL inheritance by the creator DACL

Here, we build the creator security descriptor with a DACL ❶ and keep the same inheritable parent security descriptor as in [Listing 6-23](#) ❷. In the output, we see that the ACEs from the creator's DACL have been copied to the new security descriptor ❸.

The previous two tests haven't specified any auto-inherit flags. If we specify the `DacLAutoInherited` control flag on the parent security descriptor but include no creator DACL, then the inheritance proceeds in the same way as in [Listing 6-24](#), except that it sets the inherited ACE flags.

However, something interesting happens if we specify both a creator DACL and the control flag ([Listing 6-25](#)).

```

❶ PS> $creator = New-CreatorSD -AceFlags "Inherited"
❷ PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
    -Control "DacLAutoInherited"
    -= Parent SD -=
    <Owner> : BUILTIN\Administrators
    <Group> : BUILTIN\Administrators
    <DACL> (Auto Inherited)
    Everyone: (Allowed)(None)(Full Access)
    BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

    -= Creator SD -=
    <DACL>
    NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
    NT AUTHORITY\INTERACTIVE: (Allowed)(Inherited)(Full Access)

    -= New SD -=
    <Owner> : GRAPHITE\user
    <Group> : GRAPHITE\None
    <DACL> (Auto Inherited)

```

```

❸ NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
❹ BUILTIN\Users: (Allowed)(Inherited)(Full Access)

```

Listing 6-25: Testing parent DACL inheritance when the creator DACL and the DaclAutoInherited control flag are set

In this listing, we build a creator security descriptor and set the INTERACTIVE SID ACE to include the Inherited flag ❶. Next, we run the test with the DaclAutoInherited control flag on the parent security descriptor ❷. In the output, notice that there are two ACEs. The first ACE was copied from the creator ❸, while the second is the inherited ACE from the parent ❹. [Figure 6-1](#) shows this auto-inheritance behavior.

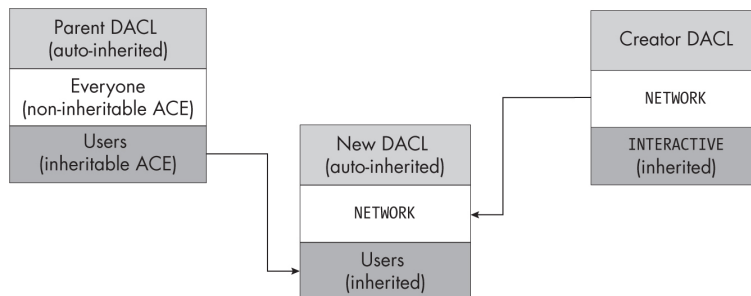


Figure 6-1: The auto-inheritance behavior when the parent and creator security descriptors are both set

When `DaclAutoInherit` is set, the new security descriptor's DACL merges the non-inherited ACEs from the creator security descriptor with the inheritable ACEs from the parent. This auto-inheritance behavior allows you to rebuild a child's security descriptor based on its parent without losing any ACEs that the user has explicitly added to the DACL. Additionally, the automatic setting of the Inherited ACE flag lets us differentiate between these explicit and inherited ACEs.

Note that normal operations in the kernel do not set the `DaclAutoInherit` flag, which is enabled only if the parent security descriptor has the `DaclAutoInherited` control flag set and the DACL isn't present. In our test, we specified a DACL, so the auto-inherit flag was not set. The Win32 APIs use this behavior, as we'll discuss later in this chapter.

If you want to suppress the merging of the explicit ACEs and the parent's inheritable ACEs, you can set the `DaclProtected` and/or `SaclProtected` security descriptor control flags. If a protected control flag is set, the inheritance rules leave the respective ACL alone, other than setting the `AutoInherited` control flag for the ACL and clearing any inherited ACE flags. In [Listing 6-26](#), we test this behavior for the DACL.

```

❶ PS> $creator = New-CreatorSD -AceFlags "Inherited" -Control "DaclProtected"
❷ PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
      -Control "DaclAutoInherited"
      -= Parent SD -=
      <Owner> : BUILTIN\Administrators
      <Group> : BUILTIN\Administrators
      <DACL> (Auto Inherited)
      Everyone: (Allowed)(None)(Full Access)
      BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

```

```

-- Creator SD --
<DACL> (Protected)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(Inherited)(Full Access)

-- New SD --
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL> (Protected, Auto Inherited)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
❸ NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

```

Listing 6-26: Testing the DacLProtected control flag

We start by generating a creator security descriptor with the `DacLProtected` flag, and setting one of the ACE's flags to `Inherited` ❶. We then create a new security descriptor with an auto-inherited parent ❷. Without the `DacLProtected` flag, the new security descriptor's DACL would have been a merged version of the creator DACL and the inheritable ACEs from the parent. Instead, we see only the creator DACL's ACEs. Also, the `Inherited` flag on the second ACE has been cleared ❸.

What if we don't know whether the parent security descriptor will have inheritable ACEs, and we don't want to end up with the default DACL? This might be important for permanent objects, such as files or keys, as the default DACL contains the ephemeral logon SID, which shouldn't really be persisted to disk. After all, reusing the logon SID could end up granting access to an unrelated user.

In this case, we can't set a DACL in the creator security descriptor; according to inheritance rules, this would overwrite any inherited ACEs. Instead, we can handle this scenario using the `DacLDefaulted` security descriptor control flag, which indicates that the provided DACL is a default.

Listing 6-27 demonstrates its use.

```

PS> $creator = New-CreatorSD -Control "DacLDefaulted"
PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
= Parent SD --
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-- Creator SD --
<DACL> (Defaulted)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

-- New SD --
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
BUILTIN\Users: (Allowed)(None)(Full Access)

```

```

PS> Test-NewSD -Creator $creator
== Parent SD ==
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(None)(Full Access)

== Creator SD ==
<DACL> (Defaulted)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

== New SD ==
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

```

Listing 6-27: Testing the DaclDefaulted flag

If the parent does not contain any inheritable DACL ACEs, the new security descriptor will use the creator's DACL instead of the default. If the parent does contain inheritable ACEs, the inheritance process will overwrite the DACL, following the rules outlined previously.

To implement similar behavior for the SACL, you can use the `SaclDefaulted` control flag. However, tokens don't contain a default SACL, so this flag is somewhat less important.

Replacing the CREATOR OWNER and CREATOR GROUP SIDs

We've seen that, during inheritance, an inherited ACE retains the same SID as the original. In some scenarios, this isn't desirable. For example, you might have a shared directory that allows any user to create a child directory. What security descriptor could you set on this shared directory so that only the creator of the child directory has access to it?

One solution would be to remove all inheritable ACEs. As a result, the new directory would use the default DACL. This would almost certainly secure the directory to prevent other users from accessing it. However, as mentioned in the previous section, the default DACL is designed for ephemeral resources, such as those in the object manager; persistent security descriptors shouldn't use it.

To accommodate features such as shared directories, the inheritance implementation supports four special creator SIDs. When a security descriptor inherits an ACE with any of these SIDs, the inheritance implementation will replace the creator SID with a specific SID from the creator's token:

CREATOR OWNER (S-1-3-0) Replaced by the token's owner

CREATOR GROUP (S-1-3-1) Replaced by the token's primary group

CREATOR OWNER SERVER (S-1-3-2) Replaced by the server's owner

CREATOR GROUP SERVER (S-1-3-3) Replaced by the server's primary group

We use the server SIDs only when creating a server security descriptor, which we'll discuss in "Server Security Descriptors and Compound ACEs" on page 213. The conversion from the creator SID to a specific SID is a one-way process: once the SID has been replaced, you can't tell it apart from a SID you set explicitly. However, if a container has inherited the ACE, it will keep the creator SID in the `InheritOnly` ACE. [Listing 6-28](#) provides an example.

```
PS> $parent = New-NtSecurityDescriptor -Type Directory
PS> Add-NtSecurityDescriptorAce $parent -KnownSid CreatorOwner
-Flags ContainerInherit, InheritOnly -Access GenericWrite
PS> Add-NtSecurityDescriptorAce $parent -KnownSid CreatorGroup
-Flags ContainerInherit, InheritOnly -Access GenericRead
PS> Format-NtSecurityDescriptor $parent -Summary -SecurityInformation Dacl
<Dacl>
❶ CREATOR OWNER: (Allowed)(ContainerInherit, InheritOnly)(GenericWrite)
CREATOR GROUP: (Allowed)(ContainerInherit, InheritOnly)(GenericRead)

PS> $token = Get-NtToken -Effective -Pseudo
❷ PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent
-Type Directory -Container
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
<Dacl>
❸ GRAPHITE\user: (Allowed)(None)(CreateObject|CreateSubDirectory|ReadControl)
CREATOR OWNER: (Allowed)(ContainerInherit, InheritOnly)(GenericWrite)
❹ GRAPHITE\None: (Allowed)(None)(Query|Traverse|ReadControl)
CREATOR GROUP: (Allowed)(ContainerInherit, InheritOnly)(GenericRead)
```

Listing 6-28: Testing creator SIDs during inheritance

We first add two ACEs with the **CREATOR OWNER** and **CREATOR GROUP** SIDs to a parent security descriptor, giving the ACEs different levels of access to make them easy to distinguish ❶. We then create a new security descriptor based on the parent, specifying that we'll use it for a container ❷. In the formatted output, we see that the user's SID has replaced the **CREATOR OWNER** SID. This SID is based on the owner SID in the token ❸. We also can see that the **CREATOR GROUP** SID has been replaced with the group SID from the token ❹.

As we've created the security descriptor for a container, we also see that there are two `InheritOnly` ACEs whose creator SID has not been changed. This behavior allows the creator SID to propagate to any future children.

Assigning Mandatory Labels

The mandatory label ACE contains the integrity level of a resource. But when we create a new security descriptor using a token whose integrity level is greater than or equal to `Medium`, the new security descriptor won't

receive a mandatory label by default. This behavior explains why we haven't seen any mandatory label ACEs in our tests so far.

If the token's integrity level is less than `Medium`, on the other hand, this label is automatically assigned to the new security descriptor, as shown in [Listing 6-29](#).

```
PS> $token = Get-NtToken -Duplicate -IntegrityLevel Low
PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
Mandatory Label\Low Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)
PS> $token.Close()
```

Listing 6-29: Assigning the mandatory label of the creator's token

In this listing, we duplicate the current token and assign it a `Low` integrity level. When we create a new security descriptor based on the token, we see that it has a mandatory label with the same integrity level.

An application can set a mandatory label ACE explicitly when creating a new resource through the creator security descriptor. However, the integrity level in the mandatory label ACE must be less than or equal to the token's integrity level; otherwise, the creation will fail, as shown in [Listing 6-30](#).

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Set-NtSecurityDescriptorIntegrityLevel $creator System
PS> $token = Get-NtToken -Duplicate -IntegrityLevel Medium
PS> New-NtSecurityDescriptor -Token $token -Creator $creator -Type Mutant
❶ New-NtSecurityDescriptor : (0xC0000061) - A required privilege is not held
  by the client.

❷ PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
  -Type Mutant -AutoInherit AvoidPrivilegeCheck
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
❸ Mandatory Label\System Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)

PS> $token.Close()
```

Listing 6-30: Assigning a mandatory label based on the creator security descriptor

First, we create a new creator security descriptor and add a mandatory label with the `System` integrity level to it. We then get the caller's token and set its integrity level to `Medium`. Because the `System` integrity level is greater than `Medium`, if we attempt to use the creator security descriptor to create a new security descriptor, the operation fails with a `STATUS_PRIVILEGE_NOT_HELD` error ❶.

To set a higher integrity level, the `SeRelabelPrivilege` privilege must be enabled on the creator token, or you must specify the `AvoidPrivilegeCheck` auto-inherit flag. In this example, we set the auto-inherit flag when creat-

ing the new security descriptor ❷. With this addition the creation succeeds, and we can see the mandatory label in the formatted output ❸.

We can make the mandatory label ACE inheritable by setting its `ObjectInherit` OR `ContainerInherit` flag. It's also possible to specify its `InheritOnly` flag, which prevents the integrity level from being used as part of an access check, reserving it for inheritance only.

Keep in mind, though, that integrity-level restrictions apply to inherited mandatory label ACEs too. The inherited ACE must have an integrity level that is less than or equal to the token's; otherwise, the security descriptor assignment will fail. Again, we can bypass this restriction with either the `SeRelabelPrivilege` privilege or the `AvoidPrivilegeCheck` auto-inherit flag.

Listing 6-31 shows an example in which a security descriptor inherits the mandatory label ACE.

```
PS> $parent = New-NtSecurityDescriptor -Type Mutant
❶ PS> Set-NtSecurityDescriptorIntegrityLevel $parent Low -Flags ObjectInherit
PS> $token = Get-NtToken -Effective -Pseudo
PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent -Type Mutant
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
❷ Mandatory Label\Low Mandatory Level: (MandatoryLabel)(Inherited)(NoWriteUp)
```

Listing 6-31: Assigning a mandatory label from a parent security descriptor through inheritance

First, we create a parent security descriptor and assign it a mandatory label ACE with a `Low` integrity level and the `ObjectInherit` flag set ❶. We then create a new security descriptor using the parent. The new security descriptor inherits the mandatory label, as indicated by the `Inherited` flag ❷.

Certain kernel object types might receive the mandatory label automatically, even if the caller's token has an integrity level greater than or equal to `Medium`. By specifying certain auto-inherit flags, you can always assign the caller's integrity level when creating a new security descriptor for the resource. These flags include `Mac1NoWriteUp`, `Mac1NoReadUp`, and `Mac1NoExecuteUp`, which auto-inherit the token's integrity level and set the mandatory policy to `NoWriteUp`, `NoReadUp`, and `NoExecuteUp`, respectively. By combining these flags, you can get the desired mandatory policy.

In the latest versions of Windows, only four types are registered to use these auto-inherit flags, as shown in **Table 6-5**.

Table 6-5: Types with Integrity Level Auto-inherit Flags Enabled

Type name	Auto-inherit flags
Process	Mac1NoWriteUp, Mac1NoReadUp
Thread	Mac1NoWriteUp, Mac1NoReadUp

Type name	Auto-inherit flags
Job	Mac1NoWriteUp
Token	Mac1NoWriteUp

We can test the behavior of these auto-inherit flags by specifying them when we create a security descriptor. In [Listing 6-32](#), we specify the Mac1NoReadUp and Mac1NoWriteUp auto-inherit flags.

```
PS> $token = Get-NtToken -Effective -Pseudo
PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
      -AutoInherit Mac1NoReadUp, Mac1NoWriteUp
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
Mandatory Label\Medium Mandatory Level: (MandatoryLabel)(None)(NoWriteUp|
NoReadUp)
```

Listing 6-32: Assigning a mandatory label by specifying auto-inherit flags

In the output, we can see a mandatory label ACE with a Medium integrity level, even though I mentioned at the start of this section that the Medium level wouldn't normally be assigned. We can also see that the mandatory policy has been set to NoWriteUp|NoReadUp, which matches the auto-inherit flags we specified.

Determining Object Inheritance

When we specify an object ACE type, such as AllowedObject, in a parent security descriptor, the inheritance rules change slightly. This is because each object ACE can contain two optional GUIDs: ObjectType, used for access checking, and InheritedObjectType, used for inheritance.

The SeAssignSecurityEx API uses the InheritedObjectType GUID in an ACE to calculate whether a new security descriptor should inherit that ACE. If this GUID exists and its value matches the ObjectType GUID, the new security descriptor will inherit the ACE. By contrast, if the values don't match, the ACE won't be copied. [Table 6-6](#) shows the possible combinations of the ObjectType parameter and InheritedObjectType and whether the ACE is inherited.

Table 6-6: Whether to Inherit the ACE Based on InheritedObjectType

ObjectType parameter specified?	InheritedObjectType in ACE?	Inherited
No	No	Yes
No	Yes	No
Yes	No	Yes

ObjectType parameter specified?	InheritedObjectType in ACE?	Inherited
Yes	Yes (and the values match)	Yes
Yes	Yes (and the values don't match)	No

I've bolded the cases in [Table 6-6](#) where inheritance doesn't happen. Note that this doesn't supersede any other inheritance decision: the ACE must have the `ObjectInherit` and/or `ContainerInherit` flag set to be considered for inheritance.

In [Listing 6-33](#), we verify this behavior by adding some object ACEs to a security descriptor and using it as the parent.

```

PS> $owner = Get-NtSid -KnownSid BuiltinAdministrators
PS> $parent = New-NtSecurityDescriptor -Type Directory -Owner $owner -Group $owner
❶ PS> $type_1 = New-Guid
PS> $type_2 = New-Guid
❷ PS> Add-NtSecurityDescriptorAce $parent -Name "SYSTEM" -Access GenericAll
-Flags ObjectInherit -Type AllowedObject -ObjectType $type_1
❸ PS> Add-NtSecurityDescriptorAce $parent -Name "Everyone" -Access GenericAll
-Flags ObjectInherit -Type AllowedObject -InheritedObjectType $type_1
❹ PS> Add-NtSecurityDescriptorAce $parent -Name "Users" -Access GenericAll
-Flags ObjectInherit -InheritedObjectType $type_2 -Type AllowedObject
PS> Format-NtSecurityDescriptor $parent -Summary -SecurityInformation Dacl
<Dacl>
NT AUTHORITY\SYSTEM: (AllowedObject)(ObjectInherit)(GenericAll) (OBJ:f5ee1953...)
Everyone: (AllowedObject)(ObjectInherit)(GenericAll)(IOBJ:f5ee1953...)
BUILTIN\Users: (AllowedObject)(ObjectInherit)(GenericAll)(IOBJ:0b9ed996...)

PS> $token = Get-NtToken -Effective -Pseudo
❺ PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent
-Type Directory -ObjectType $type_2
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
<Dacl>
❻ NT AUTHORITY\SYSTEM: (AllowedObject)(None)(Full Access)(OBJ:f5ee1953...)
❼ BUILTIN\Users: (Allowed)(None)(Full Access)

```

Listing 6-33: Verifying the behavior of the InheritedObjectType GUID

We first generate a couple of random GUIDs to act as our object types ❶. Next, we add three inheritable `AllowedObject` ACEs to the parent security descriptor. In the first ACE, we set `ObjectType` to the first GUID we created ❷. This ACE demonstrates that the `ObjectType` GUID is not considered when inheriting the ACE. The second ACE sets the `InheritedObjectType` to the first GUID ❸. The final ACE uses the second GUID ❹.

We then create a new security descriptor, passing the second GUID to the `ObjectType` parameter ❺. When we check the new security descriptor, we can see that it inherited the ACE without the `InheritedObjectType` ❻. The second ACE in the output is a copy of the ACE with an `InheritedObjectType`

GUID that matches ❶. Notice that, based on the output, the `InheritedObjectType` has been removed, as the ACE is no longer inheritable.

Having a single `ObjectType` GUID parameter is somewhat inflexible, so Windows also provides two APIs that take a list of GUIDs rather than a single GUID: the `SeAssignSecurityEx2` kernel API and the `RtlNewSecurityObjectWithMultipleInheritance` user-mode API. Any ACE in the list with the `InheritedObjectType` will be inherited; otherwise, the inheritance rules are basically the same as those covered here.

This concludes our discussion on assigning security descriptors during creation. As you've seen, the assignment process is complex, especially with regard to inheritance. We'll now discuss assigning a security descriptor to an existing resource, a considerably simpler process.

Assigning a Security Descriptor to an Existing Resource

If a resource already exists, it's not possible to set the security descriptor by calling a creation system call such as `NtCreateMutant` and specifying the `SecurityDescriptor` field in the object attributes. Instead, you need to open a handle to the resource with one of three access rights, depending on what part of the security descriptor you want to modify. Once you have this handle, you can call the `NtSetSecurityObject` system call to set specific security descriptor information. [Table 6-7](#) shows the access rights needed to set each security descriptor field based on the `SecurityInformation` enumeration.

Table 6-7: `SecurityInformation` Flags and Required Access for Security Descriptor Creation

Flag name	Description	Location	Handle access required
Owner	Set the owner SID.	Owner	<code>WriteOwner</code>
Group	Set the group SID.	Group	<code>WriteOwner</code>
Dacl	Set the DACL.	DACL	<code>WriteDac</code>
Sacl	Set the SACL (for auditing ACEs only).	SACL	<code>AccessSystemSecurity</code>
Label	Set the mandatory label.	SACL	<code>WriteOwner</code>
Attribute	Set a system resource attribute.	SACL	<code>WriteDac</code>

Flag name	Description	Location	Handle access required
Scope	Set a scoped policy ID.	SACL	AccessSystemSecurity
ProcessTrustLabel	Set the process trust label.	SACL	WriteDac
AccessFilter	Set an access filter.	SACL	WriteDac
Backup	Set everything except the process trust label and access filter.	All	WriteDac, WriteOwner, and AccessSystemSecurity

You might notice that the handle access required for setting this information is more complex than the access needed to merely query it (covered in [Table 6-1](#)), as it is split across three access rights instead of two. Rather than trying to memorize these access rights, you can retrieve them using the `Get-NtAccessMask` PowerShell command, specifying the parts of the security descriptor you want to set with the `SecurityInformation` parameter, as shown in [Listing 6-34](#).

```
PS> Get-NtAccessMask -SecurityInformation AllBasic -ToGenericAccess
ReadControl

PS> Get-NtAccessMask -SecurityInformation AllBasic -ToGenericAccess
-SetSecurity
WriteDac, WriteOwner
```

Listing 6-34: Discovering the access mask needed to query or set specific security descriptor information

To set a security descriptor, the `NtSetSecurityObject` system call invokes a type-specific security function. This type-specific function allows the kernel to support the different storage requirements for security descriptors; for example, a file must persist its security descriptor to disk, while the object manager can store a security descriptor in memory.

These type-specific functions eventually call the `SeSetSecurityDescriptorInfoEx` kernel API to build the updated security descriptor. User mode exports this kernel API as `RtlSetSecurityObjectEx`. Once the security descriptor has been updated, the type-specific function can store it using its preferred mechanism.

The `SeSetSecurityDescriptorInfoEx` API accepts the following five parameters and returns a new security descriptor:

Modification security descriptor The new security descriptor passed to `NtSetSecurityObject`

Object security descriptor The current security descriptor for the object being updated

Security information Flags to specify what parts of the security descriptor to update, described in [Table 6-7](#)

Auto-inherit A set of bit flags that define the auto-inheritance behavior

Generic mapping The generic mapping for the type being created

No kernel code uses the auto-inherit flags; therefore, the behavior of this API is simple. It merely copies the parts of the security descriptor specified in the security information parameter to the new security descriptor. It also maps any generic access to the type-specific access using the generic mapping, excluding `InheritOnly` ACEs.

Some security descriptor control flags introduce special behavior. For example, it's not possible to explicitly set `DaclAutoInherited`, but you can specify it along with `DaclAutoInheritReq` to set it on the new security descriptor.

We can test out the `RtlSetSecurityObjectEx` API using the `Edit-NtSecurityDescriptor` command, as shown in [Listing 6-35](#).

```
PS> $owner = Get-NtSid -KnownSid BuiltinAdministrators
PS> $obj_sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner
           -Group $owner
PS> Add-NtSecurityDescriptorAce $obj_sd -KnownSid World -Access GenericAll
PS> Format-NtSecurityDescriptor $obj_sd -Summary -SecurityInformation Dacl
<Dacl>
Everyone: (Allowed)(None)(Full Access)

PS> Edit-NtSecurityDescriptor $obj_sd -MapGeneric
PS> $mod_sd = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $mod_sd -KnownSid Anonymous
           -Access GenericRead
PS> Set-NtSecurityDescriptorControl $mod_sd DaclAutoInherited,
           DaclAutoInheritReq
PS> Edit-NtSecurityDescriptor $obj_sd $mod_sd -SecurityInformation Dacl
PS> Format-NtSecurityDescriptor $obj_sd -Summary -SecurityInformation Dacl
<Dacl> (Auto Inherited)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(None)(ModifyState|ReadControl)
```

Listing 6-35: Using `Edit-NtSecurityDescriptor` to modify an existing security descriptor

You can set the security for a kernel object using the `Set-NtSecurityDescriptor` command. The command can accept either an object handle with the required access or an OMNS path to the resource. For example, you could use the following commands to try to modify the object `|BaseNamedObjects|ABC` by setting a new DACL:


```
PS> $new_sd = New-NtSecurityDescriptor -Sddl "D:(A;;;GA;;;WD)"
PS> Set-NtSecurityDescriptor -Path "\BaseNamedObjects\ABC"
-SecurityDescriptor $new_sd -SecurityInformation Dacl
```

Note the “try to”: even if you can open a resource with the required access to set a security descriptor component, such as `WriteOwner` access, this doesn’t mean the kernel will let you do it. The same rules regarding owner SIDs and mandatory labels apply here as when assigning a security descriptor at creation time.

The `SeSetSecurityDescriptorInfoEx` API enforces these rules. If no object security descriptor is specified, then the API returns the `STATUS_NO_SECURITY_ON_OBJECT` status code. Therefore, you can’t set the security descriptor for a type with `SecurityRequired` set to `False`; that object won’t have a security descriptor, so any attempt to modify it causes the error.

NOTE

One ACE flag I haven’t mentioned yet is `Critical`. The Windows kernel contains code to check the `Critical` flag and block the removal of ACEs that have the flag set. However, which ACEs to deem `Critical` is up to the code assigning the new security descriptor, and APIs such as `SeSetSecurityInformationEx` do not enforce it. Therefore, do not rely on the `Critical` flag to do anything specific. If you’re using security descriptors in user mode, you can handle the flag any way you like.

What happens if you change the inheritable ACEs on a container? Will the changes in the security descriptor propagate to all existing children? In a word, no. Technically, a type could implement this automatic propagation behavior, but none do. Instead, it’s up to the user-mode components to handle it. Next, we’ll look at the user-mode Win32 APIs that implement this propagation.

Win32 Security APIs

Most applications don’t directly call the kernel system calls to read or set security descriptors. Instead, they use a range of Win32 APIs. While we won’t discuss every API you could use here, we’ll cover some of the additional functionality the APIs add to the underlying system calls.

Win32 implements the `GetKernelObjectSecurity` and `SetKernelObjectSecurity` APIs, which wrap `NtQuerySecurityObject` and `NtSetSecurityObject`. Likewise, the `CreatePrivateObjectSecurityEx` and `SetPrivateObjectSecurityEx` Win32 APIs wrap `RtlNewSecurityObjectEx` and `RtlSetSecurityObjectEx`, respectively. Every property of the native APIs discussed in this chapter applies equally to these Win32 APIs.

However, Win32 also provides some higher-level APIs: most notably, `GetNamedSecurityInfo` and `SetNamedSecurityInfo`. These APIs allow an application to query or set a security descriptor by providing a path and the type of resource that path refers to, rather than a handle. The use of a path and type allows the functions to be more general; for example, these APIs support getting and setting the security of not only files and registry keys

but also services, printers, and Active Directory Domain Services (DS) entries.

To query or set the security descriptor, the API must open the specified resource and then call the appropriate API to perform the operation. For example, to query a file's security descriptor, the API would open the file using the `CreateFile` Win32 API and then call the `NtQuerySecurityObject` system call. However, to query a printer's security descriptor, the Win32 API needs to open the printer using the `OpenPrinter` print spooler API and then call the `GetPrinter` API on the opened printer handle (as a printer is not a kernel object).

PowerShell already uses the `GetNamedSecurityInfo` API through the `Get-Acl` command; however, the built-in command doesn't support reading certain security descriptor ACEs, such as mandatory labels. Therefore, the `NtObjectManager` module implements `Get-Win32SecurityDescriptor`, which calls `GetNamedSecurityInfo` and returns a `SecurityDescriptor` object.

If you merely want to display the security descriptor, you can use the `Format-Win32SecurityDescriptor` command, which takes the same parameters but doesn't return a `SecurityDescriptor` object. [Listing 6-36](#) provides a couple of examples of commands that leverage the underlying Win32 security APIs.

```
PS> Get-Win32SecurityDescriptor "$env:WinDir"
Owner          DACL ACE Count SACL ACE Count Integrity Level
-----
NT SERVICE\TrustedInstaller 13          NONE          NONE

PS> Format-Win32SecurityDescriptor "MACHINE\SOFTWARE" -Type RegistryKey
-Summary
<Owner> : NT AUTHORITY\SYSTEM
<Group> : NT AUTHORITY\SYSTEM
<DACL> (Protected, Auto Inherited)
BUILTIN\Users: (Allowed)(ContainerInherit)(QueryValue|...)
--snip--
```

Listing 6-36: An example usage of `Get-Win32SecurityDescriptor` and `Format-Win32SecurityDescriptor`

We start by using `Get-Win32SecurityDescriptor` to query the security descriptor for the *Windows* directory, in this case `$env:WinDir`. Note that we don't specify the type of resource we want to query, as it defaults to a file. In the second example, we use `Format-Win32SecurityDescriptor` to display the security descriptor for the `MACHINE\SOFTWARE` key. This key path corresponds to the Win32 `HKEY_LOCAL_MACHINE\SOFTWARE` key path. We need to indicate that we're querying a registry key by specifying the `Type` parameter; otherwise, the command will try to open the path as a file, which is unlikely to work.

NOTE

To find the path format for every supported type of object, consult the API documentation for the `SE_OBJECT_TYPE` enumeration, which is used

to specify the type of resource in the *GetNamedSecurityInfo* and *SetNamedSecurityInfo* APIs.

The *SetNamedSecurityInfo* API is more complex, as it implements auto-inheritance across hierarchies (for example, across a file directory tree). As we discussed earlier, if you use the *NtSetSecurityObject* system call to set a file's security descriptor, any new inheritable ACEs won't get propagated to any existing children. If you set a security descriptor on a file directory with *SetNamedSecurityInfo*, the API will enumerate all child files and directories and attempt to update each child's security descriptor.

The *SetNamedSecurityInfo* API generates the new security descriptor by querying the child security descriptor and using it as the creator security descriptor in a call to *RtlNewSecurityObjectEx*, taking the parent security descriptor from the parent directory. The *DaclAutoInherit* and *SaclAutoInherit* flags are always set, to merge any explicit ACEs in the creator security descriptor into the new security descriptor.

PowerShell exposes the *SetNamedSecurityInfo* API through the *Set-Win32SecurityDescriptor* command, as shown in [Listing 6-37](#).

```
PS> $path = Join-Path "$env:TEMP" "TestFolder"
❶ PS> Use-NtObject($f = New-NtFile $path -Win32Path -Options DirectoryFile
-Disposition OpenIf) {
    Set-NtSecurityDescriptor $f "D:AIARP(A;OICI;GA;;;WD)" Dacl
}

PS> $item = Join-Path $path test.txt
PS> "Hello World!" | Set-Content -Path $item
PS> Format-Win32SecurityDescriptor $item -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
❷ Everyone: (Allowed)(Inherited)(Full Access)

PS> $sd = Get-Win32SecurityDescriptor $path
PS> Add-NtSecurityDescriptorAce $sd -KnownSid Anonymous -Access GenericAll
-Flags ObjectInherit,ContainerInherit,InheritOnly
❸ PS> Set-Win32SecurityDescriptor $path $sd Dacl
PS> Format-Win32SecurityDescriptor $item -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
Everyone: (Allowed)(Inherited)(Full Access)
❹ NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Listing 6-37: Testing auto-inheritance with *Set-Win32SecurityDescriptor*

[Listing 6-37](#) demonstrates the auto-inheritance behavior of *SetNamedSecurityInfo* for files. We first create the *TestFolder* directory in the root of the system drive, then set the security descriptor so that it contains one inheritable ACE for the *Everyone* group and has the *DaclAutoInherited* and *DaclProtected* flags set ❶. Next, we create a text file inside the directory and print its security descriptor. The DACL contains the single ACE inherited from the parent by the text file ❷.

We then get the security descriptor from the directory and add a new inheritable ACE to it for the anonymous user. We use this security descriptor to set the DACL of the parent using *Set-Win32SecurityDescriptor* ❸.

Printing the text file's security descriptor again, we now see that it has two ACEs, as the anonymous user ACE has been added ❷. If we had used `Set-NtSecurityDescriptor` to set the parent directory's security descriptor, this inheritance would not have taken place.

Because `SetNamedSecurityInfo` always uses auto-inheritance, applying a protected security descriptor control flag, such as `DaclProtected` or `SaclProtected`, becomes an important way to block the automatic propagation of ACEs.

Oddly, the API doesn't allow you to specify the `DaclProtected` and `SaclProtected` control flags directly in the security descriptor. Instead, it introduces some additional `SecurityInformation` flags to handle setting and unsetting the control flags. To set a protected security descriptor control flag, you can use the `ProtectedDacl` and `ProtectedSacl` flags for `SecurityInformation`. To unset a flag, use `UnprotectedDacl` and `UnprotectedSacl`. [Listing 6-38](#) provides examples of setting and unsetting the protected control flag for the DACL.

```
PS> $path = Join-Path "$env:TEMP\TestFolder" "test.txt"
❶ PS> $sd = New-NtSecurityDescriptor "D:(A;;GA;;;AU)"
PS> Set-Win32SecurityDescriptor $path $sd Dacl,ProtectedDacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
❷ <DACL> (Protected, Auto Inherited)
NT AUTHORITY\Authenticated Users: (Allowed)(None)(Full Access)

❸ PS> Set-Win32SecurityDescriptor $path $sd Dacl,UnprotectedDacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
❹ <DACL> (Auto Inherited)
NT AUTHORITY\Authenticated Users: (Allowed)(None)(Full Access)
Everyone: (Allowed)(Inherited)(Full Access)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Listing 6-38: Testing the ProtectedDacl and UnprotectedDacl SecurityInformation flags

This script assumes you've run [Listing 6-37](#) already, as it reuses the file created there. We create a new security descriptor with a single ACE for the *Authenticated Users* group and assign it to the file with the `ProtectedDacl` and `Dacl` flags ❶. As a result, the protected control flag for the DACL is now set on the file ❷. Note that the inherited ACEs from [Listing 6-37](#) have been removed; only the new, explicit ACE is left.

We then assign the security descriptor again with the `UnprotectedDacl` flag ❸. This time, when we print the security descriptor we can see that it no longer has the protected control flag set ❹. Also, the API restores the inherited ACEs from the parent directory and merges them with the explicit ACE for the *Authenticated Users* group.

The behavior of the command when we specify the `UnprotectedDacl` flag shows you how you can restore the inherited ACEs for any file. If you specify an empty DACL so no explicit ACEs will be merged, and additionally specify the `UnprotectedDacl` flag, you'll reset the security descriptor to

the version based on its parent. To simplify this operation, the PowerShell module contains the `Reset-Win32SecurityDescriptor` command ([Listing 6-39](#)).

```
PS> $path = Join-Path "$env:TEMP\TestFolder" "test.txt"
PS> Reset-Win32SecurityDescriptor $path Dacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
<DAcl> (Auto Inherited)
Everyone: (Allowed)(Inherited)(Full Access)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Listing 6-39: Resetting the security of a directory using `Reset-Win32SecurityDescriptor`

In this listing, we call `Reset-Win32SecurityDescriptor` with the path to the file and request that the DACL be reset. When we display the security descriptor of the file, we now find that it matches the parent directory's security descriptor, shown in [Listing 6-37](#).

THE DANGERS OF AUTO-INHERITANCE

The auto-inheritance features of the Win32 security APIs are convenient for applications, which can merely set an inheritable security descriptor to apply it to any child resources. However, auto-inheritance introduces a security risk, especially if used by privileged applications or services.

The risk occurs if the privileged application can be tricked into resetting the inherited security for a hierarchy when a malicious user has control over the parent security descriptor. For example, CVE-2018-0983 was a security vulnerability in the privileged storage service: it called `SetNamedSecurityInfo` to reset the security of a file with the path specified by the user. By using some filesystem tricks, an attacker could link the file being reset to a system file that was writable by an administrator only. However, the `SetNamedSecurityInfo` API thought the file was in a directory controlled by the user, so it reset the security descriptor based on that directory's security descriptor, granting the malicious user full access to the system file.

Microsoft has fixed this issue, and Windows no longer supports the filesystem tricks necessary to exploit it. However, there are other potential ways for a privileged service to be tricked. Therefore, if you're writing code to set or reset the security descriptor of a resource, pay careful attention to where the path comes from. If it's from an unprivileged user, make sure you impersonate the caller before calling any of the Win32 security APIs.

One final API to cover is `GetInheritanceSource`, which allows you to identify the source of a resource's inherited ACEs. One reason ACEs are marked with the `Inherited` flag is to facilitate the analysis of inherited ACEs. Without the flag, the API would have no way of distinguishing between inherited and non-inherited ACEs.

For each ACE with the `Inherited` flag set, the API works its way up the parent hierarchy until it finds an inheritable ACE that doesn't have this flag set but contains the same SID and access mask. Of course, there is no guarantee that the found ACE is the actual source of the inherited ACE, which could potentially live further up the hierarchy. Thus, treat the output of `GetInheritanceSource` as purely informational, and don't use it for security-critical decisions.

Like the other Win32 APIs, `GetInheritanceSource` supports different types. However, it's limited to resources that have a child-parent relationship, such as files, registry keys, and DS objects. You can access the API through the `Search-Win32SecurityDescriptor` command, as shown in [Listing 6-40](#).

```
PS> $path = Join-Path "$env:TEMP" "TestFolder"
PS> Search-Win32SecurityDescriptor $path | Format-Table
Name                Depth User                Access
----                -
0                   Everyone             GenericAll
0                   NT AUTHORITY\ANONYMOUS LOGON GenericAll

PS> $path = Join-Path $path "new.txt"
PS> "Hello" | Set-Content $path
PS> Search-Win32SecurityDescriptor $path | Format-Table
Name                Depth User                Access
----                -
C:\Temp\TestFolder\ 1      Everyone             GenericAll
C:\Temp\TestFolder\ 1      NT AUTHORITY\ANONYMOUS LOGON GenericAll
```

Listing 6-40: Enumerating inherited ACEs using Search-Win32SecurityDescriptor

We first call `Search-Win32SecurityDescriptor` with the path to the directory we created in [Listing 6-38](#). The output is a list of the ACEs in the resource's DACL, including the name of the resource from which each ACE was inherited and the depth of the hierarchy. We set two explicit ACEs on the directory. The output reflects this as a `Depth` value of 0, which indicates that the ACE wasn't inherited. You can also see that the `Name` column is empty.

We then create a new file in the directory and rerun the command. In this case, as you might have expected, the ACEs show that they were both inherited from the parent folder, with a `Depth` of 1.

This section covered the basics of the Win32 APIs. Keep in mind that there are clear differences in behavior between these APIs and the low-level system calls, especially regarding inheritance. When you interact with the security of resources via a GUI, it's almost certainly calling one of the Win32 APIs.

Server Security Descriptors and Compound ACEs

Let's finish this chapter with a topic I briefly mentioned when we discussed creator SIDs: server security descriptors. The kernel supports two very poorly documented security descriptor control flags for servers:

`ServerSecurity` and `DaclUntrusted`. We use these flags only when generating a new security descriptor, either at object creation time or when assigning a security descriptor explicitly. The main control flag, `ServerSecurity`, indicates to the security descriptor generation code that the caller is expecting to impersonate another user.

When a new security descriptor is created during impersonation, the owner and group SIDs will default to the values from the impersonation token. This might not be desirable, as being the owner of a resource can grant a caller additional access to it. However, the caller can't set the owner to an arbitrary SID, because the SID must be able to pass the owner check, which is based on the impersonation token.

This is where the `ServerSecurity` control flag comes in. If you set the flag on the creator security descriptor when creating a new security descriptor, the owner and group SIDs default to the primary token of the caller, and not to the impersonation token. This flag also replaces all `Allowed` ACEs in the DACL with `AllowedCompound` ACEs, the structure of which we defined back in [Chapter 5](#). In the compound ACE, the server SID is set to the owner SID from the primary token. [Listing 6-41](#) shows an example.

```

❶ PS> $token = Get-NtToken -Anonymous
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $creator -KnownSid World -Access GenericAll
PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Owner,Group,Dacl

❷ <Owner> : NT AUTHORITY\ANONYMOUS LOGON
<Group> : NT AUTHORITY\ANONYMOUS LOGON
<DACL>
Everyone: (Allowed)(None)(Full Access)

❸ PS> Set-NtSecurityDescriptorControl $creator ServerSecurity
PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Owner,Group,Dacl

❹ <Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>

❺ Everyone: (AllowedCompound)(None)(Full Access)(Server:GRAPHITE\user)

```

Listing 6-41: Testing the `ServerSecurity` security descriptor control flag

We first create a new security descriptor using the anonymous user token ❶. This initial test doesn't set the `ServerSecurity` flag. As expected, the `Owner` and `Group` default to values based on the `Anonymous` user token, and the single ACE we added remains intact ❷. Next, we add the `ServerSecurity` control flag to the creator security descriptor ❸. After calling `New-NtSecurityDescriptor` again, we now find that the `Owner` and `Group` are set to the defaults for the primary token, not to those of the `Anonymous` user token ❹. Also, the single ACE has been replaced with a compound ACE, whose server SID is set to the primary token's owner SID ❺. We'll discuss how changes to compound ACEs impact access checking in [Chapter 7](#).

The `DaclUntrusted` control flag works in combination with `ServerSecurity`. By default, `ServerSecurity` assumes that any compound ACE in the DACL is

trusted and will copy it verbatim into the output. When the `DaclUntrusted` control flag is set, all compound ACEs instead have their server SID values set to the primary token's owner SID.

If the `ServerSecurity` control flag is set on the creator security descriptor and the new security descriptor inherits ACEs from a parent, we can convert the `CREATOR OWNER SERVER` and `CREATOR GROUP SERVER` SIDs to their respective primary token values. Also, any inherited `Allowed` ACEs will be converted to compound ACEs, except for those of the default DACL.

A Summary of Inheritance Behavior

Inheritance is a very important topic to understand. [Table 6-8](#) summarizes the ACL inheritance rules we've discussed in this chapter, to help you make sense of them.

Table 6-8: Summary of Inheritance Rules for the DACL

Parent ACL	Creator ACL	Auto-inherit set	Auto-inherit not set
None	None	Default	Default
None	Present	Creator	Creator
Non-inheritable	None	Default	Default
Inheritable	None	Parent	Parent
Non-inheritable	Present	Creator	Creator
Inheritable	Present	Parent and creator	Creator
Non-inheritable	Protected	Creator	Creator
Inheritable	Protected	Creator	Creator
Non-inheritable	Defaulted	Creator	Creator
Inheritable	Defaulted	Parent	Parent

The first two columns in this table describe the state of the parent ACL and the creator ACL; the last two describe the resulting ACL, depending on whether the `DaclAutoInherit` and/or `SaclAutoInherit` flag was set. There are six ACL types to consider:

None The ACL isn't present in the security descriptor.

Present The ACL is present in the security descriptor (even if it is a NULL or empty ACL).

Non-inheritable The ACL has no inheritable ACEs.

Inheritable The ACL has one or more inheritable ACEs.

Protected The security descriptor has the `DaclProtected` OR `SaclProtected` control flag set.

Defaulted The security descriptor has the `DaclDefaulted` OR `SaclDefaulted` control flag set.

Additionally, there are four possible resulting ACLs:

Default The default DACL from the token, or nothing in the case of a SACL

Creator All ACEs from the creator ACL

Parent The inheritable ACEs from the parent ACL

Parent and creator The inheritable ACEs from the parent and explicit ACEs from the creator

When an auto-inherit flag is set, the new security descriptor will have the corresponding `DaclAutoInherited` OR `SaclAutoInherited` control flag set. Also, all ACEs that were inherited from the parent ACL will have the `Inherited` ACE flag set. Note that this table doesn't consider the behavioral changes due to object ACEs, mandatory labels, server security, and creator SIDs, which add more complexity.

Worked Examples

Let's walk through some worked examples that use the commands you've learned about in this chapter.

Finding Object Manager Resource Owners

As you've seen in this chapter, the owner of a resource's security descriptor is usually the user who created the resource. For administrators, however, it's typically the built-in *Administrators* group. The only way to set a different owner SID is to use another token group SID that has the `Owner` flag set, or to enable `SeRestorePrivilege`. Neither option is available to non-administrator users.

Thus, knowing the owner of a resource can indicate whether a more privileged user created and used the resource. This could help you identify potential misuses of the Win32 security APIs in privileged applications, or find shared resources that a lower-privileged user might write to; a privileged user could mishandle these, causing a security issue.

Listing 6-42 shows a simple example: finding object manager resources whose owner SID differs from the caller's.

```
PS> function Get-NameAndOwner {❶
    [CmdletBinding()]
    param(
        [parameter(Mandatory, ValueFromPipeline)]
        $Entry,
        [parameter(Mandatory)]
        $Root
    )

    begin {
        $curr_owner = Get-NtSid -Owner ❷
    }

    process {
        $sd = Get-NtSecurityDescriptor -Path $Entry.Name -Root $Root ❸
        -TypeName $Entry.NtTypeName -ErrorAction SilentlyContinue
        if ($null -ne $sd -and $sd.Owner.Sid -ne $curr_owner) {
            [PSCustomObject] @{
                Name = $Entry.Name
                NtTypeName = $Entry.NtTypeName
                Owner = $sd.Owner.Sid.Name
                SecurityDescriptor = $sd
            }
        }
    }
}

PS> Use-NtObject($dir = Get-NtDirectory \BaseNamedObjects) {❹
    Get-NtDirectoryEntry $dir | Get-NameAndOwner -Root $dir
}

Name                                NtTypeName  Owner                                SecurityDescriptor
----                                -
CLR_PerfMon_DoneEnumEvent           Event       NT AUTHORITY\SYSTEM                O:SYG:SYD:(A;;...
WAMACAP0;3_Read                     Event       BUILTIN\Administrators             O:SYG:SYD:(A;;...
WAMACAP0;8_Mem                      Section     BUILTIN\Administrators             O:SYG:SYD:(A;;...
--snip--
```

Listing 6-42: Finding objects in BaseNamedObjects that are owned by a different user

We first define a function to query the name and owner of an object manager directory entry ❶. The function initializes the `$curr_owner` variable with the owner SID of the caller's token ❷. We'll compare this SID with the owner of a resource to return only resources owned by a different user.

For each directory entry, we query its security descriptor using the `Get-NtSecurityDescriptor` command ❸. We can specify a path and a root directory object to the command to avoid having to manually open the resource. If we successfully query the security descriptor, and if the owner SID does not match the current user's owner SID, we return the resource's name, object type, and owner SID.

To test the new function, we open a directory (in this case, the global *BaseNamedObjects* directory ❹) and use `Get-NtDirectoryEntry` to query for all entries, piping them through the function we defined. We receive a list of resources not owned by the current user.

For example, the output includes the `WAMACAPO;8_Mem` object, which is a shared memory *Section* object. If a normal user can write to this *Section* object, we should investigate it further, as it might be possible to trick a privileged application into performing an operation that would elevate a normal user's privileges.

We can test our ability to get write access on the *Section* object by using the `Get-NtGrantedAccess` command with the `SecurityDescriptor` property of the object, as shown in [Listing 6-43](#).

```
PS> $entry
Name                NtTypeName  Owner                SecurityDescriptor
-----
WAMACAPO;8_Mem      Section      BUILTIN\Administrators O:SYG:SYD:(A;;...

PS> Get-NtGrantedAccess -SecurityDescriptor $entry.SecurityDescriptor
Query, MapWrite, MapRead, ReadControl
```

Listing 6-43: Getting the granted access for a Section object

The `$entry` variable contains the object we want to inspect. We pass its security descriptor to the `Get-NtGrantedAccess` command to return the maximum granted access for that resource. In this case, we can see that `MapWrite` is present, which indicates that the *Section* object could be mapped as writable.

The example I've shown in [Listing 6-42](#) should provide you with an understanding of how to query for any resource. You can replace the directory with a file or registry key, then call `Get-NtSecurityDescriptor` with the path and the root object to query the owner for each of these resource types.

For the object manager and registry, however, there is a much simpler way of finding the owner SID. For the registry, we can look up the security descriptor for the entries returned from the *NtObject* drive provider using the `SecurityDescriptor` property. For example, we can select the `Name` and `Owner` SID fields for the root registry key using the following script:

```
PS> ls NtKey:\ | Select Name, {$_.SecurityDescriptor.Owner.Sid}
```

We can also specify the `Recurse` parameter to perform the check recursively.

If you want to query the owner SIDs of files, you can't use this technique, as the file provider does not return the security provider in its entries. Instead, you need to use the built-in `Get-Acl` command. Here, for example, we query a file's ACL:

```
PS> ls C:\ | Get-Acl | Select Path, Owner
```

The `Get-Acl` command returns the owner as a username, not a SID. You'll have to look up the SID manually using the `Get-NtSid` command and the `Name` parameter if you need it. Alternatively, you can convert the output of the `Get-Acl` command to a `SecurityDescriptor` object used in the `NtObjectManager` module, as shown in [Listing 6-44](#).

```
PS> (Get-Acl C:\ | ConvertTo-NtSecurityDescriptor).Owner.Sid
Name                               Sid
----                               -
NT SERVICE\TrustedInstaller S-1-5-80-956008885-3418522649-1831038044-...
```

Listing 6-44: Converting Get-Acl output to a SecurityDescriptor object

We use the `ConvertTo-NtSecurityDescriptor` PowerShell command to perform the conversion.

Changing the Ownership of a Resource

Administrators commonly take ownership of resources. This allows them to easily modify a resource's security descriptor and gain full access to it. Windows comes with several tools for doing this, such as *takeown.exe*, which sets the owner of a file to the current user. However, you'll find it instructive to go through the process of changing the owner manually, so you can understand exactly how it works. Run the commands in [Listing 6-45](#) as an administrator.

```
PS> $new_dir = New-NtDirectory "ABC" -Win32Path
PS> Get-NtSecurityDescriptor $new_dir | Select {$_.Owner.Sid.Name}
$_.Owner.Sid.Name
-----
BUILTIN\Administrators

PS> Enable-NtTokenPrivilege SeRestorePrivilege
PS> Use-NtObject($dir = Get-NtDirectory "ABC" -Win32Path -Access WriteOwner) {
    $sid = Get-NtSid -KnownSid World
    $sd = New-NtSecurityDescriptor -Owner $sid
    Set-NtSecurityDescriptor $dir $sd -SecurityInformation Owner
}

PS> Get-NtSecurityDescriptor $new_dir | Select {$_.Owner.Sid.Name}
$_.Owner.Sid.Name
-----
Everyone

PS> $new_dir.Close()
```

Listing 6-45: Setting an arbitrary owner for a Directory object

We start by creating a new `Directory` object on which to perform the operations. (We'll avoid modifying an existing resource, which might risk breaking your system.) We then query the resource's current owner SID.

In this case, because we're running this script as an administrator, it's set to the *Administrators* group.

Next, we enable the `SeRestorePrivilege` privilege. We need to do this only if we want to set an arbitrary owner SID. If we want to set a permitted SID, we can skip this line. We then open the `Directory` again, but only for `WriteOwner` access.

We can now create a security descriptor with just the owner SID set to the `World` SID. To do this, we call the `Set-NtSecurityDescriptor` PowerShell command, specifying only the `Owner` flag. If you haven't enabled `SeRestorePrivilege`, this operation will fail with a `STATUS_INVALID_OWNER` status code. To confirm that we've changed the owner SID, we query it again, which confirms that it's now set to `Everyone` (the name of the `World` SID).

You can apply this same set of operations to any resource type, including registry keys and files: simply change the command used to open the resource. Whether you'll be granted `WriteOwner` access depends on the specifics of the access check process. In [Chapter 7](#), you'll learn about a few cases in which the access check automatically grants `WriteOwner` access based on certain criteria.

Wrapping Up

This chapter began with an overview of how to read the security descriptor of an existing kernel resource using the `Get-NtObjectSecurity` command. We covered the security information flags that define what parts of the security descriptors the command should read and outlined the special rules for accessing audit information stored in the SACL.

We then discussed how we can assign security descriptors to resources, either during the resource creation process or by modifying an existing resource. In the process, you learned about ACL inheritance and auto-inheritance. We also discussed the behavior of the Win32 APIs, specifically `SetNamedSecurityInfo`, and how that API implements auto-inheritance even though the kernel doesn't explicitly implement it. We concluded with an overview of the poorly documented server security descriptor and compound ACEs. In the next chapter, we'll (finally) discuss how Windows combines the token and security descriptor to check whether a user can access a resource.